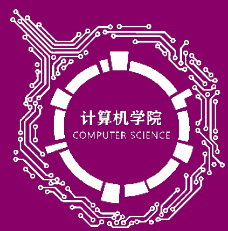


第七章 类和对象

主讲： 刘晓光 张海威
张莹 殷爱茹
沈玮 宋春瑶
李雨森 卢少平





面向对象程序设计思想



类的定义与对象的说明



构造函数与析构函数



常对象与常量成员



静态成员



友元



类与类之间的关系



类中的运算符重载



简单的数据结构设计



面向对象程序设计思想
类的定义与对象的说明
构造函数与析构函数
常对象与常量成员



☐ 结构化程序设计与面向对象程序设计
☐ 类和对象的基本概念
☐ 面向对象程序设计的特点
☐ 面向对象程序设计的过程



面向对象程序设计思想



类的定义与对象的说明



构造函数与析构函数



常对象与常量成员



静态成员



友元



面向对象程序设计思想 ☒
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

结构化程序设计与面向对象程序设计 ☒
类和对象的基本概念 ☐
面向对象程序设计的特点 ☐
面向对象程序设计的过程 ☐

面向对象的程序设计思想

结构化程序设计思想

- 以函数为核心
- 程序主体为函数定义的集合
- 以功能为研究对象

面向对象程序设计思想

- 以类为核心
- 程序主体为类定义的集合
- 以整体为研究对象



面向对象程序设计思想 ■
类的定义与对象的说明 □
构造函数与析构函数 □
常对象与常量成员 □

□ 结构化程序设计与面向对象程序设计
■ 类和对象的基本概念
□ 面向对象程序设计的特点
□ 面向对象程序设计的过程

类和对象的基本概念

什么是类

- 类是对现实世界中客观事物的抽象，通常将众多的具有相同属性的事物归纳、划分成为某个类。面向对象方法中的类，是对具有相同属性和行为的同一类对象的抽象描述，其内部包括属性（本类的数据成员）和行为（本类的成员函数）两个主要部分，即是说，类以数据为中心，把相关的一批函数组成为一体



面向对象程序设计思想 ■
类的定义与对象的说明 □
构造函数与析构函数 □
常对象与常量成员 □

□ 结构化程序设计与面向对象程序设计
■ 类和对象的基本概念
□ 面向对象程序设计的特点
□ 面向对象程序设计的过程

类和对象的基本概念

什么是对象

- 类的实例

- 如果将类看做数据类型，那么该类的对象就是相应类型的变量
- 如果将类看做某类事物的概括，那么该类的对象是类的实例



面向对象程序设计思想 ■
类的定义与对象的说明 □
构造函数与析构函数 □
常对象与常量成员 □

□ 结构化程序设计与面向对象程序设计
□ 类和对象的基本概念
■ 面向对象程序设计的特点
□ 面向对象程序设计的过程

面向对象程序设计的特点

封装性

- 将同类事物的共同属性封装为一类

继承性

- 从其它事物中继承某些属性

多态性

- 函数、运算符重载
- 虚函数



面向对象程序设计思想 ■
类的定义与对象的说明 □
构造函数与析构函数 □
常对象与常量成员 □

□ 结构化程序设计与面向对象程序设计
□ 类和对象的基本概念
■ 面向对象程序设计的特点
□ 面向对象程序设计的过程

面向对象程序设计的特点

类的主要组成

- 类的成员变量
- 类的成员函数

类的确定

- 抽象事物的共同特征
- 将事物的共同特征最为类的成员（以变量形式描述）
- 将对事物共同特征的处理设计为成员函数

面向对象程序设计过程

【例7.1】抽象公共属性

- 花色
- 点数





面向对象程序设计思想 ■
类的定义与对象的说明 □
构造函数与析构函数 □
常对象与常量成员 □

□ 结构化程序设计与面向对象程序设计
□ 类和对象的基本概念
□ 面向对象程序设计的特点
■ 面向对象程序设计的过程

面向对象程序设计过程

【例7.1】Poker类成员变量的设计

```
class Poker
{
    int number; //点数
    int suit; //花色
};
```



面向对象程序设计过程

【例7.1】Poker类成员函数的设计

- 考虑一张扑克牌可能进行的操作，对花色或者点数的操作

可以设计其它的类，使用Poker类的对象

- 拖拉机
- 斗地主
- 梭哈
-



面向对象程序设计过程

描述客观事物必须用**不同的数据类型**来描述不同的方面。

【例7.2】描述商品

- 商品名称（用**字符串**描述），该商品数量（用**整型**数描述），该商品单价（用**浮点数**描述），该商品总价（用**浮点数**描述）。
- 使用属于三种不同数据类型的四个数据成员（data member）来描述一种商品
- 商品类的函数成员需根据具体的需求进行设计，如商品出库、入库等等



面向对象程序设计思想
类的定义与对象的说明
构造函数与析构函数
常对象与常量成员



结构化程序设计与面向对象程序设计
类和对象的基本概念
面向对象程序设计的特点
面向对象程序设计的过程

面向对象程序设计过程

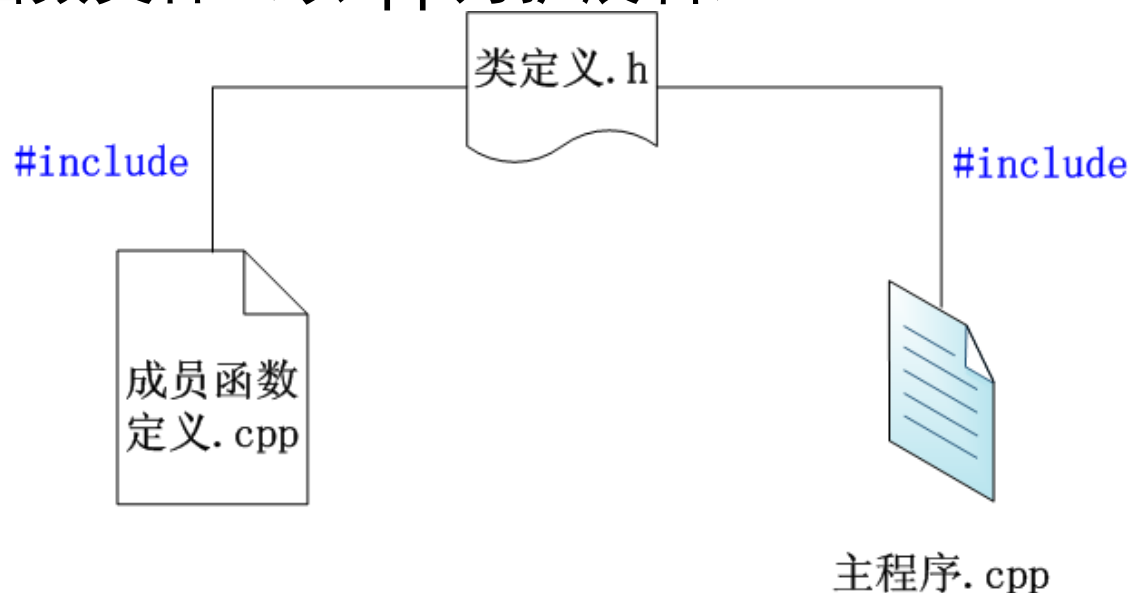
【例7.2】商品类的设计

```
class Product
{
    char * name;
    int count;
    float price;
    float total_price;
public:
    //成员函数若干
}
```

面向对象程序设计过程

面向对象程序的结构

- 类定义文件（以h为扩展名）
- 类的成员函数定义文件（以cpp为扩展名）
- 主函数文件（以cpp为扩展名）





面向对象程序设计思想
类的定义与对象的说明
构造函数与析构函数
常对象与常量成员



☐ 类的定义
☐ 成员变量与成员函数
☐ 类对象的说明
☐ this指针



面向对象程序设计思想



类的定义与对象的说明



构造函数与析构函数



常对象与常量成员



静态成员



友元



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
this指针 ☐

类的定义

理解为定义一种新的数据类型

```
class <自定义类类型名> {  
    private:  
        <各私有成员说明>;  
    public:  
        <各公有成员说明>;  
    protected:  
        <各保护成员说明>;  
    <以关键字friend开头的友元说明>;  
}; //注意类定义后面带分号
```



类的定义

类名

- 标识符，表示类的名称，理解为新的数据类型名，可以像`int`等数据类型一样说明变量（即类对象）

成员

- 按功能划分
 - 成员变量
 - 成员函数
- 按访问权限划分
 - 公有成员（由`public`标识）
 - 私有成员（由`private`标识，为默认权限）
 - 保护成员（由`protected`标识）



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
this指针 ☐

类的成员

成员变量

- 类的数据成员
- 代表该类对象含有的数据（描述属性）
- 包括普通变量、数组、指针、结构、类对象等

成员函数

- 即函数成员，对该类对象所含数据进行操作的方法
- 既可放于类定义的花括号之中，也可按类外定义方式放于之外（但要求类体内必须有其函数原型，且类定义外函数说明的前面必须用“<类名>::”来限定）
- 凡在类体中定义的函数成员均隐含为内联函数



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
`this`指针 ☐

类的成员

私有成员

- 以关键字`private`说明
- 成员不能从外部进行访问
 - 所谓类的外部是指类的定义之外以及类的成员函数定义之外

公有成员

- 以关键字`public`说明
- 成员能从外部进行访问



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
`this`指针 ☐

类的成员

保护成员

- 以关键字`protected`说明
- 类继承涉及到的概念
 - 派生类对基类成员的访问权限



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
`this` 指针 ☐

类的友元说明

友元

- 以关键字 `friend` 说明
- **不是**本类的成员，被说明为本类友元的那些函数享有本类成员函数的“权利”，即可以访问本类的私有成员



面向对象程序设计思想 ☐
类的定义与对象的说明 ☒
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
`this`指针 ☐

类定义的代码组织

`private`、`public`、`protected`以及`friend`“说明段”的顺序可以任意；每一类型的“说明段”均可以出现多次，或者一次也不出现。

若紧随左花括号后的第一个“说明段”为`private`的话，则该`private`关键字可以省略



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
`this`指针 ☐

类成员的访问权限

每种说明符（`private`、`public`、`protected`，亦称**访问限定符**）可在类体中使用多次。

访问限定符的作用域是从该说明符出现开始到下一个说明符之前或类体结束之前结束。

如果在类体起始点无访问说明符，系统默认定义为私有（`private`）。

访问限定符`private`（私有的）和`protected`（保护的）体现了类具有封装性（Encapsulation）



面向对象程序设计思想 ☐
类的定义与对象的说明 ☒
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
`this` 指针 ☐

【例7.3】设计一个类，描述梭哈游戏（ShowHand）

- 一副牌由5张扑克组成
- 判断各种牌型
 - 同花顺
 - 顺
 - 同花
 - full house
 -



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
`this`指针 ☐

【例7.3】 ShowHand类成员函数的设计

- straight flush 同花顺
- straight 顺子
- four of a kind 四张相同的牌
- full house 三张相同和二张相同的牌
- three of a kind 三张相同的牌
- two pairs 双对子
- 换牌



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
this指针 ☐

【例7.3】 ShowHand类的定义

```
class ShowHand{  
    Poker pokers[5]; //成员变量即数据成员  
public:  
    bool Straight_Flush();  
    bool Four_of_a_Kind();  
    bool Full_House();  
    bool Straight();  
    bool Flush();  
    bool Three_of_a_Kind();  
    bool Two_Pairs();  
    //根据实际需要，设计其它成员  
}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

类的定义 ☒
成员变量与成员函数 ☐
类对象的说明 ☐
`this`指针 ☐

【例7.3】 ShowHand类的成员函数之一： Flush

```
bool ShowHand::Flush()  
{  
    int i=1;  
    while(i<5) {  
        if(pokers[i-1].suit!=pokers[i].suit) {  
            return false;  
        }  
        i++;  
    }  
    return true;  
}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☒ 类的定义
☒ 成员变量与成员函数
☐ 类对象的说明
☐ `this` 指针

类的成员变量

普通变量（包括结构类型变量）

数组（包括结构类型数组）

指针（包括结构类型指针变量）

类对象

- 普通对象
- 对象数组
- 对象指针



类的成员变量

成员变量的初始化

- 类似于普通变量（数组、指针）、结构变量（数组、指针）、类对象（对象数组、对象指针）的初始化
- 可以在类成员变量说明时给出默认值，如果说明对象时未对成员变量进行初始化，可以使用默认值

【例如】

```
class Cube
{
    double side=2;
public:
    void print(){
        cout<<side<<endl;
    }
};
```



类的成员函数定义

类内定义

- 直接在类的定义内，将成员函数以“**函数定义**”的方式进行说明
 - 函数定义后，可以在“}”后加分号，也可以不加分号

类外定义

- 在类的定义内，将成员函数以“**函数原型**”的方式进行说明
- 在类的定义外，用“**限定运算符::**”对成员函数进行定义



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 类的定义
☒ 成员变量与成员函数
☐ 类对象的说明
☐ this指针

类的成员函数定义

在类的定义外，定义成员函数的格式

<返回值类型> <类名>::<函数名>(<参数表>)
{<函数体>}

【例7.3】判断“顺牌”的函数Straight在类定义外的定义为：

```
bool ShowHand::Straight()  
{  
    //函数体  
}
```



类的封装性

类把数据（事物的属性）和函数（事物的行为——操作）**封装**为一个整体。

- 成员函数可以直接使用类定义中的**任何成员**，可以处理数据成员，也可调用函数成员。
- 类是一种数据类型，定义类时系统**不为类分配存储空间**，类中的任何数据成员也不能使用关键字extern限定其存储类型。



类的对象

类可以看做用户自定义的**数据类型**

类的对象即为该类型的**变量**，当然，还可以说明该类型的

- 数组
- 指针
- 引用

对象是类的**实例**（instance）。定义一种数据类型只是告诉编译系统该数据类型的构造，并没有分配内存。类只是一个样板，以此样板可以在内存中开辟出同样结构的实例——对象。



【例7.3】 ShowHand类的成员pokers[i]为一个Poker类的对象

- 每个Poker类对象表示一张扑克牌
 - 花色不同
 - 点数不同
- 不能认为Poker类是一张扑克牌
- 每个ShowHand类的对象表示一副牌
 - 每副牌由5张牌组成



类对象的说明

普通对象（变量）

- <类名> <对象名1>,<对象名2>,....;

对象数组（数组）

- <类名> <对象数组名>[数组大小];

对象指针（指针）

- <类名> *<对象指针变量名>;

对象引用（引用）

- <类名> &<引用变量名>;



类对象的说明

【例如】各种类对象的说明示例

- 一张扑克牌

```
Poker poker;
```

- 一副扑克牌

```
Poker pokers[54];
```

- 指向一张扑克牌的指针

```
Poker *ptr_poker;
```

- 一张扑克牌的引用

```
Poker &ref_poker;
```



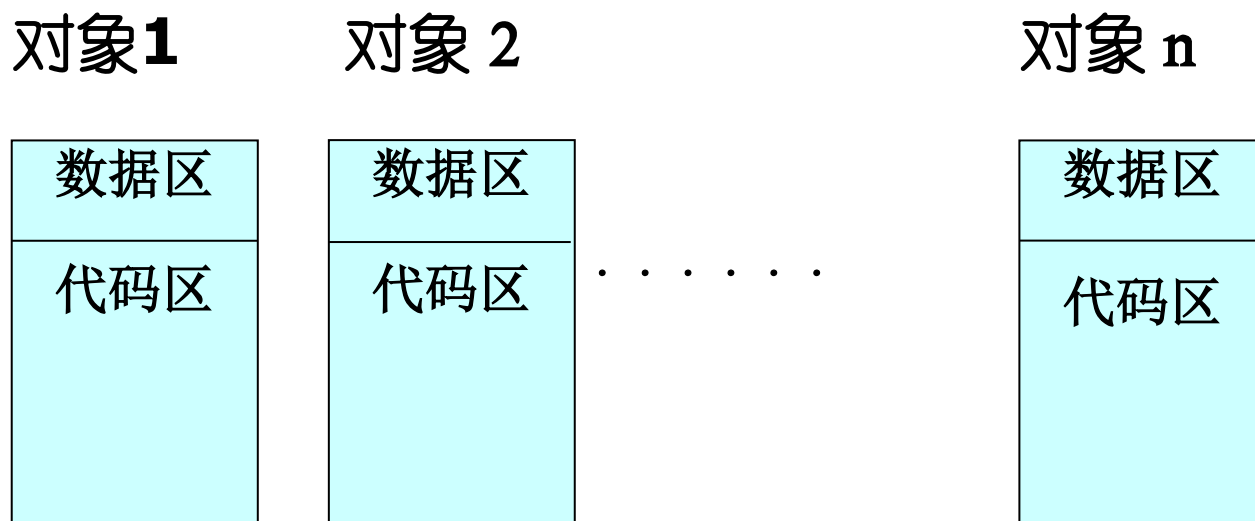
类对象的说明

除上述提到的对象数组、指向对象的指针等概念及用法外，对象还可进行如下一些操作与使用：

- 同类型的对象间可以相互赋值。
- 对象可作为函数参数（如，对象作形参，对象指针作函数参数等）。
- 函数的返回值可以是对象（或指向对象的指针）。
- 可以在一个类中说明具有类类型的成员，可以是该类的对象，亦可是其它类的对象

类对象的存储

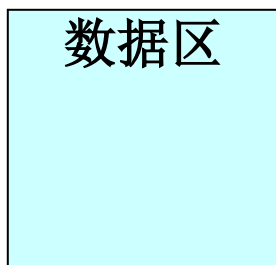
在类说明中定义函数，系统为每一个对象分配了全套的内存。数据区安放成员数据，代码区安放成员函数。



类对象的存储

在类说明外部定义函数,为每个对象分配一个数据区,代码区(放成员函数的区域)为各对象类共用。

对象1



对象 2



...

对象 n





面向对象程序设计思想 ☐
类的定义与对象的说明 ☒
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 类的定义
☐ 成员变量与成员函数
☒ 类对象的说明
☐ this指针

类对象的存储

注意：区别同一个类的各个不同的对象的属性是由数据成员决定的，不同对象的数据成员的内容是不一样的；而行为（操作）是用函数来描述的，这些操作的代码对所有对象都是一样的



对类成员的访问

对象访问类成员

- 普通类对象
 - 成员访问符 “.”
 - <对象名>.<成员变量>
 - <对象名>.<成员函数>
- 指针对象
 - 成员访问符 “->”
 - <指针对象名>-><成员变量>
 - <指针对象名>-><成员函数>
- 注意成员的访问权限



面向对象程序设计思想	☐	☐ 类的定义
类的定义与对象的说明	☐	☐ 成员变量与成员函数
构造函数与析构函数	☐	☒ 类对象的说明
常对象与常量成员	☐	☐ this指针

【例7.4】读程序，查看运行结果

```
#include <iostream>
using namespace std;
class MyClassType1 {
    private:
        int x,y;
    public:
        int getx() {return x;}
        int gety();
        void setx(int x0) {x=x0;}
        void sety(int y0);
        void displayx() {cout<<"x="<<x<<endl;}
        void displayy() {cout<<"y="<<y<<endl;}
};

int MyClassType1::gety() {return y;}
void MyClassType1::sety(int y0) {y=y0;}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☒
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 类的定义
☐ 成员变量与成员函数
☒ 类对象的说明
☐ this指针

```
void main() {  
    MyClassType1 obj1, *p, objArr[10];  
    obj1.setx(111);  
    //cout<<obj1.x<<endl;    //ERR!  
    cout<<obj1.getx()<<endl;  
    obj1.displayx();  
  
    p = new MyClassType1;  
    (*p).setx(56);  
    p->sety(78);  
    int k=(*p).getx();  
    int m=p->gety();  
    cout<<"k="<<k<<endl;  
    cout<<"m="<<m<<endl;  
}
```



```
for (int i=0; i<5; i++) {
    objArr[i].setx(i+10);
    objArr[i].sety(i+20);
}

p = &objArr[9];
while ( p>=&objArr[5] ) {
    p->setx(88);
    p->sety(99);
    p--;
}

for (i=0; i<10; i++)
    cout<<objArr[i].getx()<<"
"<<objArr[i].gety()<<endl;
}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☒
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 类的定义
☐ 成员变量与成员函数
☒ 类对象的说明
☐ this指针

程序执行后的显示结果如下：

111

x=111

k=56

m=78

10 20

11 21

12 22

13 23

14 24

88 99

88 99

88 99

88 99

88 99



this指针

类的每个成员函数中都包含的特殊的指针

- 指向当前对象，值为当前被调用的成员函数所属对象的起始地址
 - 【例如】对象a调用成员函数getValue，编译系统把对象a的地址赋给getValue函数中的this指针，this指针访问到的数据成员就是属于对象a的数据成员
 - this->value相当于a.value

```
void print ()  
{  
    int value = 0;  
    cout<<value<<endl;  
}
```

指向成员的指针

说明格式：<类型名> <类名>::*<指针变量名>;

<类型名> (<类名>::*<函数指针名>)(形参表);

- 对于已创建的对象，可将其包含的类成员（包括**非静态**成员变量和成员函数）地址赋给指向成员的指针

【例如】

```

class X
{
public:
    void f(int);
    int a;
};
void X::f(int x)
{
    a = x;
}

void main()
{
    int X::* pmi = &X::a;
    void (X:: * pmf)(int) = &X::f;
    X objx;
    objx.*pmi = 10;
    cout << objx.a << endl;
    (objx.*pmf)(5);
    cout << objx.a << endl;
}
    
```



面向对象程序设计思想
类的定义与对象的说明
构造函数与析构函数
常对象与常量成员



对象的初始化
构造函数
析构函数
拷贝构造函数



面向对象程序设计思想



类的定义与对象的说明



构造函数与析构函数



常对象与常量成员



静态成员



友元



对象的初始化

例子

```

class A {
    int x;
    int y;
    ...
};
        
```



A a;
 A b(5, 6);
 b.add();

A::add(~~xxx~~)
 { return x+y; }



对象初始化的含义

对象在使用之前必须进行初始化

- 如果一个类的数据成员是公有的，那么其对象的初始化与一般变量，结构变量或变量数组的初始化没有什么区别。
- **【例如】**

```
class address{  
    public:  
        long telenum;  
        char addr[30];  
};
```

```
class person{  
    public:  
        char name[15];  
        int age;  
        address paddr;  
};
```

```
person p1={"Zhang Hua",23,{2475096, "NanKai  
University"}};
```

对象初始化的方式

直接初始化

➤ 采用初始化列表的方式，类似于对数组、结构类型变量进行初始化的方法

使用构造函数

➤ 默认构造函数
➤ 自定义构造函数
➤ 重载构造函数



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☒ 构造函数
☐ 析构函数
☐ 拷贝构造函数

构造函数

A a0, b0, c0,

用来对类对象进行初始化

函数名与类名相同，无函数返回类型说明。

可有多多个构造函数（即构造函数可以重载，各自的参数表不相同）

若某个类定义中没有给出任一个显式的构造函数的话，则系统自动给出一个缺省的（隐式的）如下形式的构造函数：

<类名>() {}

- 此函数无参，且什么事情也不做

} 无参
|
| 什么都不做



默认构造函数

<类名>() {}

- 不在类定义中进行说明，也不给出函数的定义
- 当类定义中不包含显式的构造函数时，由系统自动给出，并在说明类对象时自动调用

- **【例如】**类address的定义：

```
class address{  
    public:  
        long telenum;  
        char addr[30];  
};
```

- 初始化时，使用默认的构造函数



自定义构造函数

必须在类定义内进行说明，在类定义内或外给出函数的定义

函数名和返回值类型与默认构造函数相同

- 函数可以有参数，也可以无参数

【例如】

```
class address{  
    public:  
        long telenum;  
        char addr[30];  
        ☐ address(long, char[]) ; //构造函数原型  
};
```

A @;

对象成员

自定义构造函数

address类的构造函数定义

```
address::address(long t, char add[]) {
    telenum = t; //赋初值
    strcpy(addr, add); //赋初值
}
```

另一种写法（用参数初始化表的形式）

```
address::address(long t, char add[]) {
    telenum(t); //也可以写为telenum{t}
    strcpy(addr, add); //赋初值
}
```

① 函数调用形式



自定义默认构造函数

类中包含自定义构造函数时，默认构造函数失效。如果仍想让对象被默认构造，可以采用如下方式

- 显示自定义默认构造函数

```
address() {}
```

- 使用 **default** 关键字，表示默认构造函数

```
address () = default;
```



构造函数初始化类对象

对类对象进行初始化时

- 判断有无自定义构造函数
 - 有自定义构造函数，默认构造函数失效
 - 用自定义构造函数初始化对象
 - 用默认构造函数初始化对象，但须在类定义时显示给出：
`<类名>() = default;`
 - 无自定义构造函数
 - 用默认构造函数初始化对象



构造函数初始化类对象

初始化方式

- **<类名> <对象名>;**
 - 未定义任何构造函数，使用默认构造函数
 - 定义了无参构造函数，此时默认构造函数失效
 - 避免了函数调用的二义性
 - 定义了含参数的构造函数，但是参数的初始值已经给出，这类构造函数可以看做与默认构造函数形式相同，此时默认构造函数同样失效
- **<类名> <对象名> (<实参表>);**
 - 定义了带有参数的构造函数



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☒ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☐ 拷贝构造函数

【例7.5】自定义类MyClassType1中设有三个显式的构造函数。

- 在说明MyClassType1的类对象时，系统将根据实参的多少去自动调用相应的构造函数。
- 类定义中没出现显式的析构函数(意味着使用系统隐含的什么事也不做的析构函数)



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☒ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☐ 拷贝构造函数

```
#include <iostream>
using namespace std;
class MyClassType1{
    private:
        int x,y;
    public:
        MyClassType1() { x=0; y=0; }
        MyClassType1(int x0) { x=x0; y=0; }
        MyClassType1(int x0, int y0);
        void display(){
            cout<<"x="<<x<<" , y="<<y<<endl;
        }
};
```



```
MyClassType1::MyClassType1(int x0, int y0) {  
    x=x0;  y=y0; }  
  
int main() {  
    MyClassType1 obj1; //用无参构造函数初始化  
    MyClassType1 obj2(21), obj3(31,32);  
    obj1.display();  
    obj2.display();  
    obj3.display();  
}
```

程序执行后的显示结果为：

x=0, y=0

x=21, y=0

x=31, y=32



【例7.6】用户自定义cylinder类并对该类进行使用

- 自定义cylinder圆柱体类，它具有私有数据成员r与h，表示圆柱体的底圆半径和高；并提供构造函数、计算底圆周长、底圆面积以及圆柱体体积的公有成员函数。并编制主函数，对cylinder类进行使用：说明cylinder类对象，输入底圆半径与圆柱体的高，而后负责计算出该圆柱体的表面积与体积并将结果显示出来



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☒ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☐ 拷贝构造函数

```
#include <iostream>
using namespace std;
const double PI = 3.1415926;

class cylinder {
    double r, h;
public:
    cylinder ( double r0 , double h0 ) { r=r0;
h=h0; }
    double circum();
    double area();
    double volume();
};
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☒ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☐ 拷贝构造函数

```
double cylinder::circum() {  
    return 2 * PI * r;  
}
```

```
double cylinder::area() {  
    return PI * r * r;  
}
```

```
double cylinder::volume() {  
    return PI * r * r * h;  
}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☒ 构造函数
☐ 析构函数
☐ 拷贝构造函数

```
int main () {  
    double r, h;  
    cout<<"Input r and h: ";  
    cin>>r>>h;  
    cylinder obj(r,h);  
    double a, v;  
    a = 2*obj.area() + obj.circum()*h;  
    v = obj.volume();  
    cout<<"area="<<a<<" ,    volume="<<v<<endl;  
}
```

程序执行后，屏幕显示结果为：

Input r and h: 1.1 10

area=76.7177, volume=38.0133



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☒ 构造函数
☐ 析构函数
☐ 拷贝构造函数

【例7.6】说明

- 自定义某一个类时，先“抽取”也即先抽象出该类要处理哪些数据，将它们设定为该类的数据成员 -- 如半径r与高h；而后进一步考虑要对这些数据进行一些什么样的处理，从而设定类成员函数 -- 如circum、area以及volume成员函数
- 在主调函数main中，必须通过类对象来对公有成员函数circum、area以及volume进行调用



【例7.7】反序输出问题

- 从键盘输入10个int型数，而后按输入的相反顺序输出它们。使用自定义类（类型），在类的私有数据成员中存放数据而后输出

```
#include <iostream>
using namespace std;
class Cla {
    int dat[10];
public:
    void dataIn();
    void reverseOut();
};
```



```
void Cla::dataIn() {
    cout<<"input 10 integers:"<<endl;
    for(int i=0; i<10; i++)
        cin>>dat[i];
}

void Cla::reverseOut() {
    cout<<"---- The result ----"<<endl;
    for(int i=9; i>=0; i--)
        cout<<dat[i]<<" ";
    cout<<endl;
}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☒ 构造函数
☐ 析构函数
☐ 拷贝构造函数

```
int main() {  
    Cla obj1;  
    obj1.dataIn();  
    obj1.reverseOut();  
}
```

程序执行后的输出结果为：

input 10 integers:

1 2 3 4 5 6 7 8 9 10

---- The result ----

10 9 8 7 6 5 4 3 2 1



对象数组的初始化

将对象数组看作一组同类型的对象，分别对每个数组元素进行初始化

- 利用初始化列表

【例如】

```
Poker poker[5] = {  
    Poker(11, "spade"),  
    Poker(10, "diamond"),  
    Poker(13, "heart"),  
    Poker(12, "club"),  
    Poker(1, "heart"),  
};
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☒ 构造函数
☐ 析构函数
☐ 拷贝构造函数

对象指针的初始化

通常使用动态内存分配运算符，生成动态对象，
用该动态对象初始化对象指针

【例如】

```
Poker *poker = new Poker(10, "club");
```



构造函数的进一步讨论

使用**explicit**关键字

- C++标准支持**带有一个参数的构造函数**，将参数类型隐式转换为相应的类类型。

【例如】

```
#include <iostream>
using namespace std;
class Cube
{
private:
    double side;
public:
    Cube(double aSide);
    void TestFunction(Cube aCube);
};
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☒ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☐ 拷贝构造函数

```
Cube::Cube(double aSide) :side{aSide}
{
    cout << "Call the constructor to set side to " << aSide << endl;
}
void Cube::TestFunction(Cube aCube)
{
    cout << aCube.side << endl;
}
int main()
{
    Cube c1(7.0);
    Cube c2(3.0);
    c1.TestFunction(50.0);
    c1.TestFunction(c2);
}
```

程序运行结果：

```
Call the constructor to set side to 7
Call the constructor to set side to 3
Call the constructor to set side to 50
50
3
```

去掉隐式转换机制，在构造函数说明时，可使用**explicit**关键字



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☒ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☐ 拷贝构造函数

```
#include <iostream>
using namespace std;
class Cube
{
private:
    double side;
public:
    explicit Cube(double aSide);
    void TestFunction(Cube aCube);
};
```

运行程序时会出现编译错误：

“**void Cube::TestFunction(Cube aCube)**”: 无法将参数1从“double”转换为“Cube”



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☒ 构造函数
☐ 析构函数
☐ 拷贝构造函数

构造函数的进一步讨论

委托构造函数

- 类中包含多个构造函数
- 一个构造函数在其初始化列表中调用了另一个构造函数，即将构造工作委托给另一个构造函数

```
class Box
{
double length;
double width;
double height;
public:
Box(double lv, double wv, double hv):length{lv}, width{wv}, height{hv}
{}
Box(double side):Box(side, side, side)
{}
};
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☒
常对象与常量成员 ☐

☐ 对象的初始化
☐ 构造函数
☒ 析构函数
☐ 拷贝构造函数

析构函数

在撤销对象占用的内存之前完成一些清理工作
在对象的生存期结束时使用

- 当对象退出其说明区域， 或使用delete释放动态对象时， 系统自动调用其析构函数。

析构函数的函数名：

- 函数名为：~<类名>

无函数返回类型说明且无参数。

析构函数可以由用户自定义



析构函数

一个类只可有一个析构函数，也可以缺省

- 没有重载的析构函数

若某个类定义中没有用户自定义的析构函数，则系统使用默认的析构函数，形式为：

`~<类名>() {}`

- 此函数什么事情也不做



构造函数与析构函数的执行顺序

先构造的后析构，后构造的先析构

【例7.8】分析下述程序执行后的结果

- 类String具有一个显式的构造函数与一个显式的析构函数。在说明String的类对象时，系统将自动调用其构造函数。在main函数结束时（类对象生命期也将结束时），系统将自动调用其析构函数。
- 对象“诞生”时，在构造函数中通过new分配了动态空间（系统资源），对象“死亡”时，在析构函数中应通过delete来释放所申请到的动态空间。



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☒	☒ 析构函数
常对象与常量成员	☐	☐ 拷贝构造函数

```
#include <iostream>
#include <string>
using namespace std;
class String {
    char * text;
public:
    String( char * str );
    ~String();
    void printStr() {cout<<text<<endl;}
};
String::String( char * str ) {
    cout<<"enter 'String::String',
str=>"<<str<<endl;
    text = new char[strlen(str)+1];
    strcpy( text, str );
}
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☒	☒ 析构函数
常对象与常量成员	☐	☐ 拷贝构造函数

```
String::~~String() {  
    cout<<"enter 'String::~~String',  
    text=>"<<text<<endl;  
    delete[] text;  
}
```

```
void main() {  
    String str1("a1d11");  
    String str2("s22g22");  
    str1.printStr();  
    str2.printStr();  
    cout<<"ending main!"<<endl;  
}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☒
常对象与常量成员 ☐

☐ 对象的初始化
☐ 构造函数
☒ 析构函数
☐ 拷贝构造函数

程序执行后的显示结果如下：

```
enter 'String::String', str=>a1d11
enter 'String::String', str=>s22g22
a1d11
s22g22
ending main!
enter 'String::~~String', text=>s22g22
enter 'String::~~String', text=>a1d11
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☐ 构造函数
☒ 析构函数
☐ 拷贝构造函数

修改主函数

```
void main() {  
    String str1("a1d11");  
    String str2 = str1;  
    str1.printStr();  
    str2.printStr();  
    cout<<"ending main!"<<endl;  
}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☐ 构造函数
☒ 析构函数
☐ 拷贝构造函数

C:\WINDOWS\system32\cmd.exe

```
enter 'String::String', str=>ald11
ald11
ald11
ending main!
enter 'String::~String', text=>ald11
enter 'String::~String', text=>葺葺葺葺葺葺
葺葺葺葺葺葺
请按任意键继续. . .
```

Microsoft Visual C++ Runtime Library



Debug Assertion Failed!

Program: D:\02_工作\高级语言程序设计
\Prog201902\Debug\Prog201902.exe
File: minkernel\crt\src\appcrt\heap\debug_heap.cpp
Line: 904

Expression: _CrtIsValidHeapPointer(block)

For information on how your program can cause an assertion failure, see the Visual C++ documentation on asserts.

(Press Retry to debug the application)

中止(A)

重试(R)

忽略(I)



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☐ 构造函数
☐ 析构函数
☒ 拷贝构造函数

拷贝构造函数

一种特殊的构造函数，具有一般构造函数的特性
只含有一个形参，且为本类对象的引用

拷贝构造函数的原型为：

`<类名>(<类名>&);`

作用是使用一个已存在的对象去初始化另一个正在创建的对象

类对象间的一般赋值，由拷贝构造函数实现（通过深拷贝或浅拷贝的方式）



拷贝构造函数

系统会自动生成缺省的拷贝构造函数，实现对象间的拷贝（浅拷贝）

- 使用下列形式的说明语句
 - <类名> <对象名2> (<对象名1>) ;
 - <类名> <对象名2> = <对象名1>;
- 对象作为函数的赋值参数
- 函数的返回值为类的对象

当程序出现上述三类情况时，**自动调用**拷贝构造函数

- 赋值语句**不自动调用**



拷贝构造函数

显式拷贝构造函数（深拷贝）

- 在某些情况下，必须在类定义中给出显式拷贝构造函数，以实现用户指定的拷贝功能（深拷贝）
 - 假设在某类的普通构造函数中分配并使用了某些系统资源，而
且在该类的析构函数中释放了这些资源。如果执行“浅拷贝”，
使两个对象使用相同的系统资源，调用析构函数将会两次释
放相同的资源而导致错误。
 - 给出显式的拷贝构造函数，可以在实现拷贝的过程中，为“被
拷贝”的对象分配新的系统资源，避免了重复释放资源的错误



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☒ 拷贝构造函数

【例7.9】默认拷贝构造函数产生的问题

//MyClass.h

```
class MyClass
{
    char *name;
public:
    MyClass(char *);
    ~MyClass();
    void print();
};
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☒ 拷贝构造函数

```
//MyClass.cpp
#include<iostream>
#include<string>
using namespace std;
#include"MyClass.h"
MyClass::MyClass(char *n) {
    name = new char[strlen(n)+1];
    strcpy_s(name, strlen(n)+1, n);
}
MyClass::~MyClass() {
    delete []name;
}
void MyClass::print() {
    cout<<"Member name="<<name<<endl;
}
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☒ 拷贝构造函数

```
//Main.cpp
#include<iostream>
#include"MyClass.h"
using namespace std;
int main() {
    char *p = new char[20];
    cin>>p;
    MyClass MyObj(p);
    MyClass MyObj1=MyObj; //执行拷贝构造函数
    MyObj.print();
    MyObj1.print();
}
```

运行结果：

abcde

Member name=abcde

Member name=abcde





面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☐ 构造函数
☐ 析构函数
☒ 拷贝构造函数

【例7.9】自定义拷贝构造函数

//MyClass.h

```
class MyClass
{
    char *name;
public:
    MyClass(char *);
    MyClass(MyClass&);
    ~MyClass();
    void print();
};
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☒ 拷贝构造函数

```
//MyClass.cpp
```

```
#include<iostream>
```

```
#include<string>
```

```
using namespace std;
```

```
#include "MyClass.h"
```

```
MyClass::MyClass(char *n) {
```

```
    name = new char[strlen(n)+1];
```

```
    strcpy_s(name, strlen(n)+1, n);
```

```
}
```

```
//自定义拷贝构造函数，实现深拷贝
```

```
MyClass::MyClass(MyClass& CopyObj) {
```

```
    name = new char[strlen(CopyObj.name)+1];
```

```
    strcpy_s(name, strlen(CopyObj.name)+1, CopyObj.name);
```

```
}
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☒ 拷贝构造函数

```
MyClass::~~MyClass() {  
    delete [] name;  
}  
void MyClass::print() {  
    cout<<"Member name="<<name<<endl;  
}
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☒ 拷贝构造函数

```
//Main.cpp
#include<iostream>
#include"MyClass.h"
using namespace std;
int main(){
    char *p = new char[20];
    cin>>p;
    MyClass MyObj(p);
    MyClass MyObj1=MyObj; //执行拷贝构造函数
    MyObj.print();
    MyObj1.print();
}
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☒ 拷贝构造函数

【例7.9】自定义拷贝构造函数无法解决“三种情况”之外的对象赋值问题

```
#include<iostream> //Main.cpp
#include "MyClass.h"
using namespace std;
void main() {
    char *p = new char[20];
    cin>>p;
    MyClass MyObj(p), MyObj1("lmnop");
    MyClass TempObj = MyObj; //执行拷贝构造函数
    MyObj = MyObj1; //赋值语句，未执行拷贝构造函数
    MyObj1 = TempObj;
    MyObj.print(); MyObj1.print();
}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

☐ 对象的初始化
☐ 构造函数
☐ 析构函数
☒ 拷贝构造函数

【例7.9】自定义拷贝构造函数无法解决“三种情况”之外的对象赋值问题，两种解决方案：

- 重载赋值运算符 “=”
- 自定义成员函数，实现对象赋值

```
//MyClass.h
```

```
class MyClass
```

```
{
```

```
    char *name;
```

```
public:
```

```
    MyClass(char *);
```

```
    MyClass(MyClass&);
```

```
void CopyMyClassObj(MyClass);
```

```
//自定义成员函数，实现对象赋值
```

```
MyClass operator=(MyClass&);
```

```
//重载赋值运算符
```

```
~MyClass();
```

```
void print();
```

```
};
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☒ 拷贝构造函数

```
//MyClass.cpp
#include<iostream>
#include<string>
#include"MyClass.h"
using namespace std;
MyClass::MyClass(char *n)
{
    name = new char[strlen(n)+1];
    strcpy(name,n);
}
MyClass::MyClass(MyClass& CopyObj)
{
    name = new char[strlen(CopyObj.name)+1];
    strcpy(name,CopyObj.name);
}
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☒ 拷贝构造函数

```
void MyClass::CopyMyClassObj(MyClass temp) {
    name =new char[strlen(temp.name)+1];
    strcpy(name,temp.name);
}
MyClass MyClass::operator=(MyClass &temp) {
    name = new char[strlen(temp.name)+1];
    strcpy(name,temp.name);
    return *this;
}
MyClass::~MyClass() {
    delete []name;
}
void MyClass::print() {
    cout<<"Member name="<<name<<endl;
}
```



面向对象程序设计思想	☐	☐ 对象的初始化
类的定义与对象的说明	☐	☐ 构造函数
构造函数与析构函数	☐	☐ 析构函数
常对象与常量成员	☐	☒ 拷贝构造函数

```
#include<iostream> //Main.cpp
#include "MyClass"
using namespace std;
void main() {
    char *p = new char[20];
    cin>>p;
    MyClass MyObj(p); //执行构造函数
    MyClass MyObjSwap("lmnop"); //执行构造函数
    MyClass MyObj1 = MyObj; //执行拷贝构造函数
    MyObj = MyObjSwap;
    //未执行拷贝构造函数，执行赋值运算符重载函数
    MyObjSwap = MyObj1;
    //MyObj.CopyMyClassObj(MyObjSwap); //调用成员函数
    //MyObjSwap.CopyMyClassObj(MyObj1); //调用成员函数
    MyObj.print();
    MyObj1.print();
    MyObjSwap.print();
}
```

练习7.1

定义一个学生类（Student），私有数据成员包括：姓名、性别、年龄、成绩；函数成员包括打印学生的信息（printInfo）

- 请自定义类的构造函数和析构函数
- 在主函数中定义学生类的对象数组并初始化
- 根据学生成绩排序并输出学生信息





面向对象程序设计思想
类的定义与对象的说明
构造函数与析构函数
常对象与常量成员



☐ 常对象
☐ 常量成员
☐ 常量数据成员
☐ 常量函数成员



面向对象程序设计思想



类的定义与对象的说明



构造函数与析构函数



常对象与常量成员



静态成员



友元

常对象

类类型的const变量

说明格式：

const <类名> <常对象名> (<实参表>);

构成常对象的任何成员变量（不包括由**mutable**修饰的）都不能被修改，但是可以读取成员变量值

常对象不能够调用任何成员函数

- 成员函数中能够访问成员变量，有可能间接改变成员变量的值



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☐

常对象 ☒
常量成员 ☐
常量数据成员 ☐
常量函数成员 ☐

```
#include<iostream>
using namespace std;
class Box
{
public:
    double length = 1;
    double width = 2;
    double height = 3;
    Box() {}
    void print() {
        cout << "length=" << length << endl;
        cout << "width=" << width << endl;
        cout << "height=" << height << endl;
    }
};
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☒

常对象 ☒
常量成员 ☐
常量数据成员 ☐
常量函数成员 ☐

```
void main()  
{  
    Box myBox;  
    myBox.print();  
    const Box myBox_const;  
    cout << myBox_const.length << endl;  
    myBox_const.width = 4; //error  
    myBox_const.print(); //error  
}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☒

☐ 常对象
☒ 常量成员
☐ 常量数据成员
☐ 常量函数成员

类的常量成员

由关键字`const`修饰的类成员说明称为类的常量成员

- 常量数据成员
- 常量函数成员



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☒

☐ 常对象
☐ 常量成员
☒ 常量数据成员
☐ 常量函数成员

常量数据成员

类的常量数据成员必须进行初始化，而且只能通过**构造函数的成员初始化列表**的方式进行。

在对象被创建以后，其常量数据成员的值就不允许被修改（只可读取其值，但不可进行改变）



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☒

☐ 常对象
☐ 常量成员
☒ 常量数据成员
☐ 常量函数成员

【例7.10】读下面的程序，给出程序执行后的屏幕输出结果

```
#include<iostream>
using namespace std;
class cnstCla {
    const int conMbr; //类中的常量数据成员
public:
    cnstCla(int i):conMbr(i) { //初始化列表处给出初值
    }
    void printCon() {
        cout<<"conMbr = "<<conMbr<<endl;
    }
    int getCon() {
        return conMbr;
    }
}
```



面向对象程序设计思想
类的定义与对象的说明
构造函数与析构函数
常对象与常量成员

☐ 常对象
☐ 常量成员
☒ 常量数据成员
☐ 常量函数成员

```
void processCon() {
    cout<<"-- In processCon --"<<endl;
    int x=2*conMbr+1;
    cout<<"x = 2*conMbr+1 = "<<x<<endl;
    //conMbr++; //ERR! 不能更改常量数据成员conMbr的值
}
};

void main() {
    cnstCla ob1(123), ob2(88);
    //以123和66为初值调用构造函数，给对象的常量数据成员赋值
    ob1.printCon();
    cout<<"ob2.getCon() = "<<ob2.getCon()<<endl;
    ob2.processCon();
}
```



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☒

☐ 常对象
☐ 常量成员
☒ 常量数据成员
☐ 常量函数成员

程序执行后，屏幕显示结果为：

```
conMbr = 123
```

```
ob2.getCon() = 88
```

```
-- In processCon --
```

```
x = 2*conMbr+1 = 177
```



常量函数成员

常量函数成员只有权读取相应对象的内容，但无权修改它们。

- 说明格式

<类型说明符> <函数名> (<参数表>) **const**;

- 定义格式

<类型说明符> <函数名> (<参数表>) **const**

{ <函数体> }

常对象**可以调用**常量函数成员



面向对象程序设计思想 ☐
类的定义与对象的说明 ☐
构造函数与析构函数 ☐
常对象与常量成员 ☒

☐ 常对象
☐ 常量成员
☐ 常量数据成员
☒ 常量函数成员

将【例7.10】中的print成员函数改写为：

```
void print() const
{
    cout << "length=" << length << endl;
    cout << "width=" << width << endl;
    cout << "height=" << height << endl;
}

void main()
{
    const Box myBox_const;
    myBox_const.print(); //correct!
}
```

静态成员 ☒
友元 ☐
类与类之间的关系 ☐
类中的运算符重载 ☐

☐ 类的静态成员
☐ 静态数据成员
☐ 静态函数成员



面向对象程序设计思想



类的定义与对象的说明



构造函数与析构函数



常对象与常量成员



静态成员



友元

类的静态成员

由关键字`static`修饰的类成员说明称为类的静态成员。

- 静态数据成员
- 静态函数成员

类的静态成员为其所有对象所共享，不管有多少对象，静态成员都只有一份存于公用内存中

静态数据成员

类的静态数据成员为该类的所有对象所共享。

在类定义中对静态数据成员进行说明

必须在类外文件作用域中的某个地方对静态数据成员赋初值（且要按如下的格式）：

＜类型＞ ＜类名＞::＜静态数据成员＞ = ＜初值＞；

访问静态数据成员有三种方式

- ＜类名＞::＜静态数据成员名＞
- ＜对象名＞ . ＜静态数据成员名＞
- ＜对象指针＞ -> ＜静态数据成员名＞

静态函数成员

类的静态函数成员没有this 指针，从而无法处理不同调用者对象的各自数据成员值。通常情况下，类的静态函数只处理类的静态数据成员。

对类的静态成员的访问通常为

<类名>::<静态函数成员调用>

也可以为

<对象名> . <静态函数成员调用>

<对象指针> -><静态函数成员调用>

【例7.11】读下面的程序，给出程序执行后的屏幕显示结果

```
#include <iostream>
using namespace std;
class claA {
public:
    double x,y;
    static int num;
    //公有静态数据成员 -- 供所有对象“共享”
    //用于记录已通过构造函数生成了多少个对象。
    claA() {
        x=0; y=0;
        num++; //每生成一个对象，num加1
    }
}
```

```
claA(double x0, double y0) {  
    x=x0; y=y0;  
    num++;  
}  
static void staFun() {  
    //静态函数成员，输出静态数据成员num的当前值  
    cout<<"current_num="<<num<<endl;  
}  
};  
int claA::num=0;  
//必须在类外（使用类名限定）初始化静态数据成员
```

```
void main() {  
    claA obj(1.2, 3.4), *p; //调用一次构造函数  
  
    cout<<"claA::num="<<claA::num<<endl;  
    claA::staFun();  
    cout<<"obj.num="<<obj.num<<endl;  
    obj.staFun();  
    cout<<endl;  
  
    claA A[3]; //将三次调用其构造函数  
    cout<<"claA::num="<<claA::num<<endl;  
    claA::staFun();  
    cout<<endl;  
}
```

```
p = new claA(5.6, 7.8); //调用一次构造函数
cout<<"claA::num="<<claA::num<<endl;
claA::staFun();
cout<<"p->num="<<p->num<<endl;
p->staFun();
}
```

程序执行后，屏幕显示结果为：

```
claA::num=1
current_num=1
obj.num=1
current_num=1
```

```
claA::num=4  
current_num=4
```

```
claA::num=5  
current_num=5  
p->num=5  
current_num=5
```

注意：

- 将claA类中的数据成员x、y以及num都说明为public公有型的，是为了在主调函数main中可以直接存取它们而使程序简单化，否则（对非公有型的数据成员），在类外存取它们时还要设立类似于getx()那样的公有成员函数。

静态成员 ☐
友元 ☐
类与类之间的关系 ☐
类中的运算符重载 ☐

☐ 友元的基本概念
☐ 友元函数
☐ 友元类



面向对象程序设计思想



类的定义与对象的说明



构造函数与析构函数



常对象与常量成员



静态成员



友元

友元的基本概念

一种类成员访问权限

- 能够访问类的任何数据成员

在类的定义中，以关键字**friend**标识

- 出现在函数说明语句前，表示该函数为类的友元函数
 - 一个函数可以同时说明为多个类的友元函数
- 出现在类名之前，表示该类为类的友元类

友元函数

将普通函数说明为某类的友元函数

- 该函数定义在类外
- 说明格式为：

friend <返回值类型> <函数名>(<参数表>);

- **【例如】**在类A的定义中，将函数Func说明为类A的友元函数：

friend void Func ();

友元函数

将其它类的成员函数说明为某类的友元函数

- 前提是已有其它类的定义
- 说明格式为：

friend <返回值类型> <类名>::<函数名> (<参数表>);

- **【例如】**在类A的定义中，将类B的成员函数说明为类A的友元函数：

friend void B::Func ();

- 之前已经对类B进行了定义或说明

友元函数

友元函数**不是**类的成员函数，在函数体中访问对象的成员，必须用对象名加运算符“.”加对象成员名。但友元函数可以访问类中的所有成员，一般函数只能访问类中的公有成员。

- 友元函数不受类中的访问权限关键字限制，可以把它放在类的公有、私有、保护部分，但结果一样。
- 某类的友元函数的作用域并非该类域。如果该友元函数是另一类的成员函数，则其作用域为另一类的类域，否则与一般函数相同。

【例7.12】读程序，分析运行结果

```
#include<iostream>
#include<cmath>
using namespace std;
class pixel{
    int x,y;
public:
    pixel(int x0, int y0){
        x=x0;
        y=y0;
    }
    void printxy(){
        cout<<"pixel: ("<<x<<" , "<<y<<" ) "<<endl;
    }
    friend double getLen(pixel p1, pixel p2);
    //友元函数（原型）
};
```

```
double getLen(pixel p1, pixel p2){ //友元函数在类体外定义
    double xd=double(p1.x-p2.x); //使用pixel类的私有成员x
    double yd=double(p1.y-p2.y);
    return sqrt(xd*xd+yd*yd);
}

void main() {
    pixel p1(0,0), p2(10,10);
    p1.printxy();
    p2.printxy();
    cout<<"getLen(p1,p2)="<<getLen(p1,p2)<<endl;
}
```

程序执行后的显示结果如下：

```
pixel: (0,0)
pixel: (10,10)
getLen(p1,p2)=14.1421
```

【例7.13】 自定义一个示意性的复数类型complex，其中说明了两个友元函数，使用该类可以完成复数的加法以及对复数的输出。并编制了主函数，说明complex类对象，对定义的各函数进行调用

- 使用友元函数的注意：
 - 参加运算的所有运算分量（如类对象）必须显式地列在友元函数的参数表中。
 - 调用友元函数时根本不通过类对象

```
#include<iostream>
using namespace std;
class complex {
    double real;
    double imag;
public:
    complex();
    complex(double r, double i);
    friend complex addCom(complex c1, complex c2);
    //友元函数, 复数c1 + c2 (二参数对象相加)
    friend void outCom (complex c);
    //友元函数, 输出类对象c的有关数据 (c为参数对象)
};
```

```
complex::complex() {  
    real=0; imag=0;  
}  
complex::complex(double r, double i) {  
    real=r; imag=i;  
}  
complex addCom(complex c1, complex c2) {  
    complex c;  
    c.real=c1.real+c2.real;  
    c.imag=c1.imag+c2.imag;  
    return c;  
}  
void outCom (complex com) {  
    cout<<" ("<<com.real<<" , "<<com.imag<<" ) ";  
}
```

```
void main() {  
    complex c1(1,2), c2(3,4), res;  
    res=addCom(c1, c2); //调用友元函数时不通过类对象  
    outCom(c1);  
    cout<<" + ";  
    outCom(c2);  
    cout<<" = ";  
    outCom(res);  
    cout<<endl;  
}
```

程序执行后，屏幕显示结果为：

$(1, 2) + (3, 4) = (4, 6)$

【例7.14】 自定义一个示意性复数类型complex，其中说明了几个类成员函数，使用该类可以完成复数的加法以及对复数的输出。并编制了主函数，说明complex类对象，对定义的各函数进行调用，以验证它们的正确性。使用成员函数的注意：

- 总以当前调用者对象（*this）作为该成员函数的隐式第一运算分量；若所定义的运算多于一个运算对象时，才将其余运算对象显式地列在该成员函数的参数表中。
- 调用类成员函数时必须通过类对象。

```
#include<iostream>
using namespace std;
class complex {
    double real;
    double imag;
public:
    complex();
    complex(double r, double i);
    complex addCom(complex c2); //类成员函数
    //调用者对象与参数对象c2相加
    void outCom (); //类成员函数
    //输出调用者对象的有关数据
};
```

```
complex::complex() {  
    real=0;    imag=0;  
}  
complex::complex(double r, double i) {  
    real=r;    imag=i;  
}  
complex complex::addCom(complex c2) {  
    complex c;  
    c.real=real+c2.real;  
    c.imag=imag+c2.imag;  
    return c;  
}  
void complex::outCom () {  
    cout<<" ("<<real<<", "<<imag<<") ";  
}
```

```
void main() {  
    complex c1(1, 2), c2(3, 4), res;  
    res=c1.addCom(c2); //调用成员函数必须通过类对象  
    c1.outCom();  
    cout<<" + ";  
    c2.outCom();  
    cout<<" = ";  
    res.outCom();  
    cout<<endl;  
}
```

程序执行后，屏幕显示结果为：

$(1, 2) + (3, 4) = (4, 6)$

友元类

将一个类B说明为另一个类A的友元类，类B中的所有函数都是类A的友元函数，可以访问类A中的所有成员

说明方式：

friend <类名>;

- 【例如】

```
class A{  
    .....  
    friend B;  
    .....  
};
```

友元类

友元类的关系是单向的，如果说明类B是类A的友元类，不等于类A也是类B的友元类

友元类的关系不能传递，如果类B是类A的友元类，而类C是类B的友元类，不等于类C是类A的友元类

除非确有必要，一般不把整个类说明为友元类，而把成员函数说明为友元函数

友元的概念**破坏**了类的封装性，但有助于数据共享，能够提高程序的效率

静态成员 ☐
友元 ☐
类与类之间的关系 ☐
类中的运算符重载 ☐

☐ 类与类之间的关系概述
☐ 类的对象成员
☐ 类嵌套



类与类之间的关系



类中的运算符重载



简单的数据结构设计

类与类之间的关系

封装的实现主要是通过把若干数据和函数成员组织到一个类中完成的。

问题的另一方面就是被封装起来的各个类之间是如何发生联系的，C++语言为类和对象之间的联系提供了许多方式，主要有：

- 一个类的对象作为另一个类的成员
- 一个类的成员函数作为另一个类的友元
- 一个类定义在另一个类的说明中，即类的嵌套
- 一个类作为另一个类的派生类

类的对象成员

自定义类中的数据成员可以是另一个类的类对象——靠类定义中的对象说明来指定这样的数据成员，意味着在一个“大对象”中包含着属于另外一个类的“小对象”

类定义中的对象说明与居于在类外的对象说明语句不同，后者意味着对象的创建；类中的对象成员说明并不直接与对象的创建和初始化相联系，它要等所在的类的对象被创建时（通过构造函数）一同被创建。

【例7.15】类line的定义中，其数据成员start与end为另一个称为pixel的类对象。负责创建类line对象的构造函数还要负责一同创建出类line所包含的对象成员start与end

```
#include<iostream>
using namespace std;
class pixel {
    int x,y;
public:
    pixel() {x=0;y=0;}
    pixel(int x0, int y0) { x=x0; y=y0;}
};
```

```
class line{
    pixel start,end;
    int color;
public:
    line(int sx,int sy,int ex,int ey,int col):
        start(sx,sy),end(ex,ey){
        color=col;
    };
};

void main(){
    line line1(20,20,100,20,2),
    line2(20,20,20,100,1);
}
```

【例7.15】分析

类line 有两个对象成员start 和end，它们是pixel 类的对象。当在程序中创建line 类的对象时，必须同时创建pixel 类的两个对象：

line line1 (20, 20, 100, 20, 2), line2 (20, 20, 20, 100, 1);

首先用实参20, 20对line1.start 进行初始化，用实参100, 20对line1.end 初始化，再进行line1.color=2，完成line1 的初始化，其次进行line2 的初始化。

类的对象成员的初始化

在类的构造函数中，使用成员初始化符表初始化该类的对象成员，

成员初始化符表的格式为：

:<成员初始化符>, ..., <成员初始化符>

成员初始化符有两种形式：

- <对象成员> (<实参表>)
 - 对象成员所属类的构造函数实参表，用于初始化对象成员
- <普通成员变量> (<初值>)

包含对象成员的类对象构造

在定义（生成）一个含有对象成员的类对象时，它的构造函数被系统调用，这时将首先按照初始化符表来依次执行各对象成员的构造函数，完成各对象成员的初始化工作，而后执行本类的构造函数体。析构函数的调用顺序恰好与之相反。

如果初始化符表中没有对象成员的显式初始化，则调用**无参构造函数**初始化类对象成员

类的嵌套

一个类的说明包含在另一个类说明中，即为类的嵌套

```
class CC{  
    class C1 {...} ;  
public:  
    class C2 {...} ;  
    C1 f1 (C2) ; ...  
};
```

- 其中类C1 是类CC 的私有嵌套类，类CC 外的函数中不能使用它；类C2 是类CC 的公有嵌套类，可以在CC 类外使用，但类名应为CC::C2

类的嵌套

在main主函数中：

```
C1 a,b;
```

```
C2 a,b;
```

```
CC::C1 a,b;
```

– 都是非法的

```
CC::C2 a,b;
```

– 是合法的，它说明了两个嵌套类C2 的对象。嵌套类的使用并不方便，不宜多用

静态成员 ☐
友元 ☐
类与类之间的关系 ☐
类中的运算符重载 ☒

☐ 类中的运算符重载概述
☐ 友元方式重载运算符
☐ 成员方式重载运算符
☐ 类中运算符重载的进一步讨论



类与类之间的关系



类中的运算符重载



简单的数据结构设计

类中的运算符重载

大多数运算符实际上是一种特殊的函数

- 【例如】

- 运算符形式: `a+b`

- 函数形式:

- 定义函数 `int add( int x, int y)`

- 函数调用 `add(a,b)`

类中的运算符重载

运算符重载是函数重载的扩充

- C++允许重新定义某些运算符的作用及其实现（功能），可以给运算符扩充添加新用法（“老用法”依然可行）。
- 例:可“借用”一批老运算符（如，`*`，`+`，`-`，`&`，`>=`，`>`等），来表示并实现集合set类对象的交、并、差、元素属于、包含、真包含等运算，而后则程序中就可按如下的方式来使用它们：

```
set s1,s2,s3,s4,s5;  
s3=s1*s2; s4=s1+s2;  
s5=s1-s2; if (3&s1) ...; if (s1>=s2) ...;  
if (s1>s2) ...;
```

类中的运算符重载

运算符重载的定义是一个函数定义过程

- **【例如】**通过友元函数方式定义“&”，通过类成员函数方式定义“+”：

```
class Set {  
    int elems [maxcard];  
    int card;  
public:  
    ...  
    friend bool operator & (int, Set);  
    Set operator + (Set S2);  
    ...  
};
```

类中的运算符重载

允许使用如下两种不同方式来定义运算符重载函数

- 以类的友元函数方式定义
 - 所有运算分量必须显式地列在本友元函数的参数表中，而且这些参数类型中至少要有一个应该是说明该友元的类类型或是对该类的引用
- 以类的公有成员函数方式定义
 - 总以当前调用者对象(*this)作为该成员函数的隐式第一运算分量，若所定义的运算多于一个运算对象时，才将其余运算对象显式地列在该成员函数的参数表中

类中的运算符重载

对重载运算符的限制

- 被用户重载的运算符，其优先级、结合性、以及运算分量个数都必须与系统中的本原运算符相一致。
- 如下5个运算符不可重载：

`.` `::` `?:` `.*` `sizeof`

- 只能以类成员而不能以友元身份重载的运算符：

`=` `( )` `[ ]` `->` `typedef`

- 不可自创新的运算符。

【例7.16】 自定义一个称为point的类，其对象表示平面上的一个点(x,y)，并通过友元方式对该类重载二目运算符“==”、“+”、“-”、“^”和一目运算符“-”，用来判断两个对象是否相等，求出两个对象的和、差、距离，以及将组成一个对象的数值取反

- 以两种方式进行运算符重载

【例7.16】使用友元方式重载运算符

```
#include <iostream>
#include <cmath>
using namespace std;
class point {
    double x,y;
public:
    point(double x0=0, double y0=0);
    friend point operator-(point pt); //重载函数
    friend point operator+(point pt1, point pt2);
    friend point operator-(point pt1, point pt2);
    friend bool operator==(point pt1, point pt2);
    friend double operator^(point pt1, point pt2);
    void display();
};
```

```
point::point (double x0, double y0) {  
    x=x0;    y=y0;  
}  
point operator - (point pt) {  
    point temp;  
    temp.x=-pt.x;  
    temp.y=-pt.y;  
    return temp;  
}  
point operator + (point pt1, point pt2) {  
    point temp;  
    temp.x=pt1.x+pt2.x;  
    temp.y=pt1.y+pt2.y;  
    return temp;  
}
```



```
point operator - (point pt1, point pt2) {  
    ...  
}  
bool operator == (point pt1, point pt2) {  
    return( (pt1.x==pt2.x) && ... );  
}  
double operator ^ (point pt1, point pt2) {  
    double d1, d2, d;  
    d1=pt1.x-pt2.x;  
    d2=...;  
    d=...;  
    return (d);  
}
```

```

void point::display () {
    cout <<" ( "<<x<<" , "<<y<<" ) "<<endl;
}

void main() {
    point s0, s1(1.2, -3.5), s2(-1, 2.8), s3(6, 6);
    point s4, s5;
    cout<<"s0=";    s0.display();
    cout<<"s1=";    s1.display();
    cout<<"s2=";    s2.display();
    cout<<"s3=";    s3.display();
    s0=s1+s2;        //将调用 “operator +”函数

```



```
cout<<"s0=s1+s2=";    s0.display();  
s4=s1-s2;  
cout<<"s4=s1-s2=";    s4.display();  
s5=-s1;  
cout<<"s5=-s1=";      s5.display();  
if (s1==s1)          cout<<"s1==s1"<<endl;  
cout<<"s2^s3="<<(s2^s3)<<endl;  
}
```

程序执行后，屏幕显示结果为：

`s0=( 0, 0 )`

`s1=( 1.2, -3.5 )`

`s2=( -1, 2.8 )`

`s3=( 6, 6 )`

`s0=s1+s2=( 0.2, -0.7 )`

`s4=s1-s2=( 2.2, -6.3 )`

`s5=-s1=( -1.2, 3.5 )`

`s1==s1`

`s2^s3=7.69675`

【例7.16】使用类成员方式重载运算符

```
#include<iostream>
#include<cmath>
using namespace std;
class point {
    double x,y;
public:
    point (double x0=0, double y0=0);
    point operator - ();
    point operator + (point pt2);
    point operator - (point pt2);
    bool operator == (point pt2);
    double operator ^ (point pt2);
    void display();
};
```



```
point::point (double x0, double y0) {  
    x=x0;    y=y0;  
}  
point point::operator - () {  
    //在类体外定义, 使用 “<类名>::” 限定  
    ...  
}  
point point::operator + (point pt2) {  
    point temp;  
    temp.x=x+pt2.x;  
    temp.y=y+pt2.y;  
    return temp;  
}
```

```
point point::operator - (point pt2) {  
    ...  
}  
bool point::operator == (point pt2) {  
    return ( (x==pt2.x) && ... );  
}  
double point::operator ^ (point pt2) {  
    ...  
}  
void point::display () {  
    cout <<" ( "<<x<<" , "<<y<<" )"<<endl;  
}
```

```

void main() {
    point s0, s1(1.2, -3.5), s2(-1, 2.8), s3(6, 6);
    point s4, s5;
    cout<<"s0=";          s0.display();
    cout<<"s1=";          s1.display();
    cout<<"s2=";          s2.display();
    cout<<"s3=";          s3.display();
    s0=s1+s2;
    cout<<"s0=s1+s2=";    s0.display();
    s4=s1-s2;
    cout<<"s4=s1-s2=";    s4.display();
    s5=-s1;
    cout<<"s5=-s1=";      s5.display();
    if(s1==s1)            cout<<"s1==s1"<<endl;
    cout<<"s2^s3="<<(s2^s3)<<endl;
}

```

程序执行后，屏幕显示结果为：

`s0=( 0, 0 )`

`s1=( 1.2, -3.5 )`

`s2=( -1, 2.8 )`

`s3=( 6, 6 )`

`s0=s1+s2=( 0.2, -0.7 )`

`s4=s1-s2=( 2.2, -6.3 )`

`s5=-s1=( -1.2, 3.5 )`

`s1==s1`

`s2^s3=7.69675`

【例7.17】利用运算符重载实现集合set类型

- 在Set类中重载运算符 '+'、 '-'、 '*'、 '<'、 '<='、 '&'、 '=='、 '!='
- 使用友元函数

```
#include <iostream>
using namespace std;
const int maxcard=20;
class Set {
    int elems [maxcard];
    int card;
public:
    Set (void) {card=0;};
    void print();
    friend bool operator & (int, Set);
    friend bool operator == (Set, Set);
    friend bool operator != (Set, Set);
    friend Set operator + (Set, int);
    friend Set operator + (Set, Set);
    friend Set operator - (Set, int);
    friend Set operator * (Set, Set);
    friend bool operator < (Set, Set);
    friend bool operator <= (Set, Set);
};
```



```
bool operator & (int elem, Set s) {  
    ...  
}  
bool operator == (Set s1, Set s2) {  
    ...  
}  
...  
void Set::print () {  
    ...  
}
```

```
void main() {  
    Set s,s1,s2,s3,s4;  
    for (int i=0;i<10;i++) {  
        s=s+i;  
        s1=s1+2*i;  
        s2=s2+3*i;  
    }  
    cout<<"s=";    s.print();  
    cout<<"s1=";    s1.print();  
    cout<<"s2=";    s2.print();  
    for (i=0;i<5;i++) {  
        s=s-i;  
        s1=s1-i;  
        s2=s2-i;  
    }  
}
```

```
cout<<"After RmvElem(0->4) , s=" ;
s.print() ;
cout<<"After RmvElem(0->4) , s1=" ;
s1.print() ;
cout<<"After RmvElem(0->4) , s2=" ;
s2.print() ;
s3=s*s1 ;
s4=s+s2 ;
cout<<"s3=s*s1=" ;    s3.print() ;
cout<<"s4=s+s2=" ;    s4.print() ;
if (s3==s4)
    cout<<"SET s3=s4 " << endl ;
else
    cout<<"SET s3!=s4 " << endl ;
```

```
if (s3==s3)
    cout<<"SET  s3=s3  "<<endl;
else
    cout<<"SET  s3!=s3  "<<endl;

if (s4<=s3)
    cout<<"SET  s3  contains  s4  "<<endl;
else
    cout<<"SET  s3  do not contains  s4  "<<endl;

if (s2<=s4)
    cout<<"SET  s4  contains  s2  "<<endl;
else
    cout<<"SET  s4  do not contains  s2  "<<endl;
}
```

程序执行后的显示结果如下：

`s={0,1,2,3,4,5,6,7,8,9}`

`s1={0,2,4,6,8,10,12,14,16,18}`

`s2={0,3,6,9,12,15,18,21,24,27}`

`After RmvElem(0->4), s={5,6,7,8,9}`

`After RmvElem(0->4), s1={6,8,10,12,14,16,18}`

`After RmvElem(0->4), s2={6,9,12,15,18,21,24,27}`

`s3=s*s1={6,8}`

`s4=s+s2={5,6,7,8,9,12,15,18,21,24,27}`

`SET s3!=s4`

`SET s3=s3`

`SET s3 do not contains s4`

`SET s4 contains s2`

练习7.2

将【例7.17】关于运算符重载的程序改写为使用类成员函数实现。注意：有些运算符只能够用一种方式重载

运算符重载的进一步讨论

赋值运算符重载

复合赋值运算符重载

插入和提取运算符重载

增量和减量运算符重载

下标运算符重载

函数调用运算符重载

赋值运算符重载

将一个对象赋值给另一个对象

- 默认复制赋值运算符（默认赋值运算符重载函数）
 - 按位赋值（浅拷贝）
- 自定义赋值运算符重载函数
 - 只能以成员方式重载
 - 格式：

```
<类名>& operator=(const <类名>& <形参对象名>)  
{  
    . . . . . ;  
    return *this;  
}
```

赋值运算符重载

- 使用const保证形参不被修改
- 使用引用保证形参、this对象和返回值具有一致性，形参相当于赋值运算的右值，返回值相当于赋值运算的左值

```
Point& Point::operator=(const Point & p)
{
    x=p.x;
    y=p.y;
    return *this;
}
```

复合赋值运算符重载

定义+=、-=、*=等复合赋值运算符的重载函数

- 定义+、-、*等运算符重载函数
- 定义赋值运算符=的重载函数
- 利用上面的函数，定义复合赋值运算符重载函数

```
Point& Point::operator+=(const Point & p)
{
    *this = (*this) + p;
    return *this;
}
```

插入和提取运算符重载

插入运算符重载

```
ostream& operator<<(ostream& out, const <
类名> <对象名>)
{
    out<<对象名.成员<<...
    return out;
}
```

插入和提取运算符重载

提取运算符重载

```
istream& operator>>(istream& in, const <类  
名>& <对象名>)  
{  
    in>>对象名.成员>>... ;  
    return in;  
}
```

增量和减量运算符重载

增量和减量运算，本质上仍包含赋值运算，此外，还有前缀增量（减量）和后缀增量（减量）

- 单目运算，无参数
- 返回值类型仍为本类的引用，返回值为*this
- 通过增加无名参数区分前缀和后缀

<类名>& operator++ (); //前缀增量

<类名>& operator-- (); //前缀减量

<类名>& operator++ (int); //后缀增量

<类名>& operator-- (int); //后缀减量

增量和减量运算符重载

```
Point& operator++()  
{  
    ++x;  
    ++y;  
    return *this;  
}  
  
Point& operator++(int)  
{  
    Point temp = *this;  
    ++(*this);  
    return temp;  
}
```

下标运算符重载

下标运算符[]主要用于访问数组元素，可以重载
下标运算符访问链表等线性结构的元素

```
class List
```

```
{
```

```
...
```

```
    Node operator[] (int index) ;
```

```
}; //list[4];
```

下标运算符重载

```
Node List::operator[] (int index)
{
    int count = 0;
    Node *curr = head;
    while (curr != nullptr) {
        if (index == count)
            return *curr;
        count++;
        curr = curr->next;
    }
    return nullptr;
}
```

函数调用运算符重载

函数调用运算符(`()`), 使得类对象通过(`()`)运算符具有函数的功能, 因此如果某个类定义了(`()`)的重载函数, 该类的对象也可称为**函数对象**, 或者称为**仿函数** (functor)

- 函数对象, 其本质是类对象, 可以作为参数进行传递, 因此, 提供了一种将函数作为另一个函数的参数的方法 (另一种是函数指针做参数)
- (`()`)运算符, 不限制运算分量的数量

函数调用运算符重载

```
class Volume
{
public:
    double operator() (double x, double y, double z)
    {
        return x*y*z;
    }
};

int main()
{
    Volume volume;
    double room = volume(16,12,8.5);
}
```



类与类之间的关系



类中的运算符重载

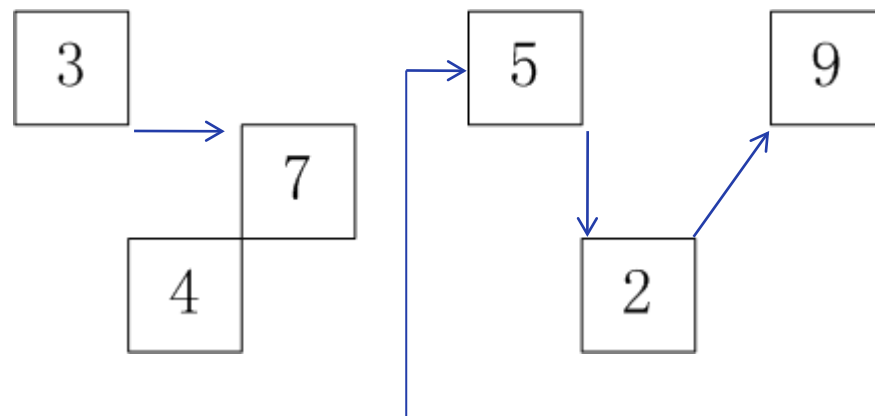
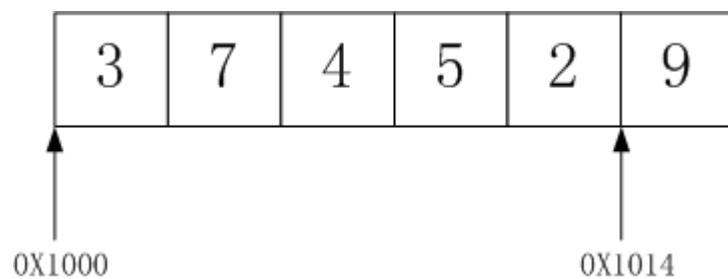


简单的数据结构设计

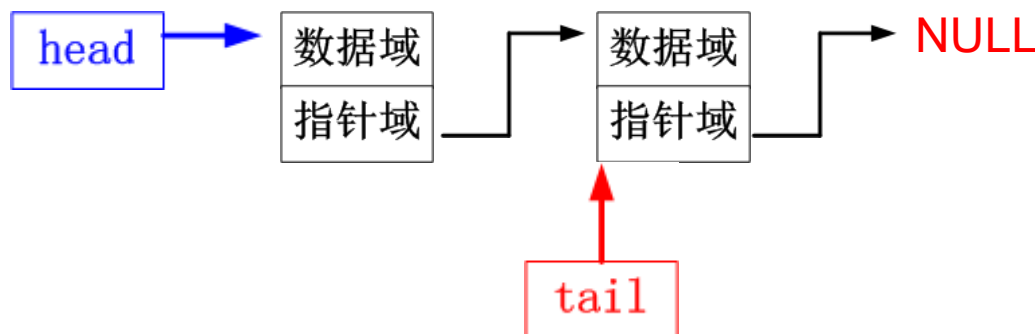
链表 (List)

链表的定义

- 重要的线性结构
- 功能与数组类似
- 与数组的区别
 - 数组存放数据的地址是连续的
 - 链表存放数据的地址是不连续的



链表类型定义



节点类

- 数据域
 - 保存链表中的数据，可以是基本类型或类类型
- 指针域
 - 指向下一个节点所在的地址

链表类

- 头指针
 - 指向链表第一个节点所在的地址
- 尾指针
 - 指向链表最后一个节点所在的地址

链表节点类

```
class Node
{
public:
    int num;
    Node* next;
    Static int TotalCount; //统计链表节点的数量
    Node(int n);
    ~Node();
};
```

链表类

```
class List
{
private:
    Node* head;
    Node* tail;
    int nodeCount; //链表节点的数量
public:
    List();
    void Insert(int n); //插入节点
    void Remove(int n); //删除节点
    void Find(int n); //查找节点
    void Print(); //打印链表的数据项
};
```

链表的主要操作

建立链表

遍历链表

显示链表

插入节点

删除节点

查找节点

建立链表

建立并初始化节点类对象

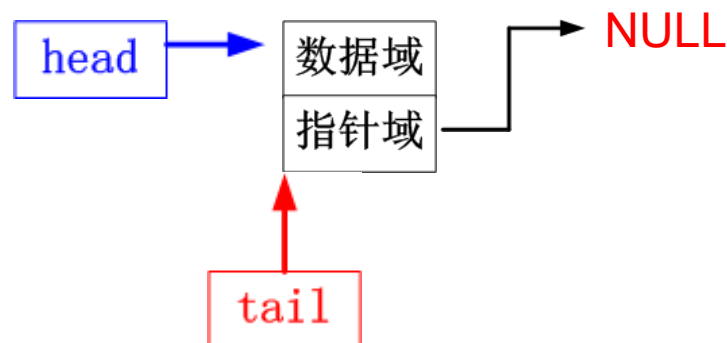
```
Node ListNode(node_num);
```

//构造函数代码略

头指针、尾指针均指向该对象

```
Node *head = &ListNode;
```

```
Node *tail = head;
```



遍历链表、显示链表数据

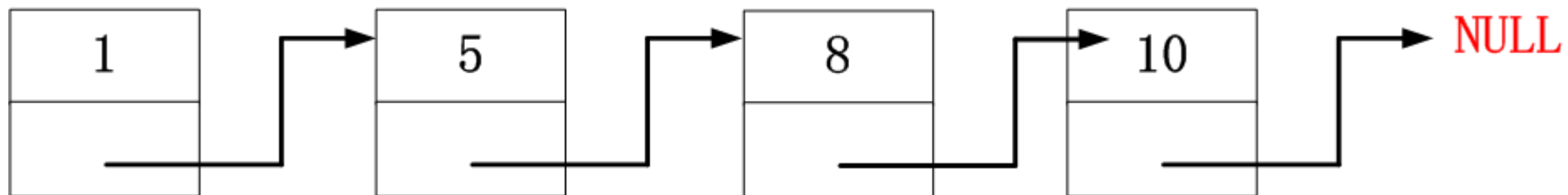
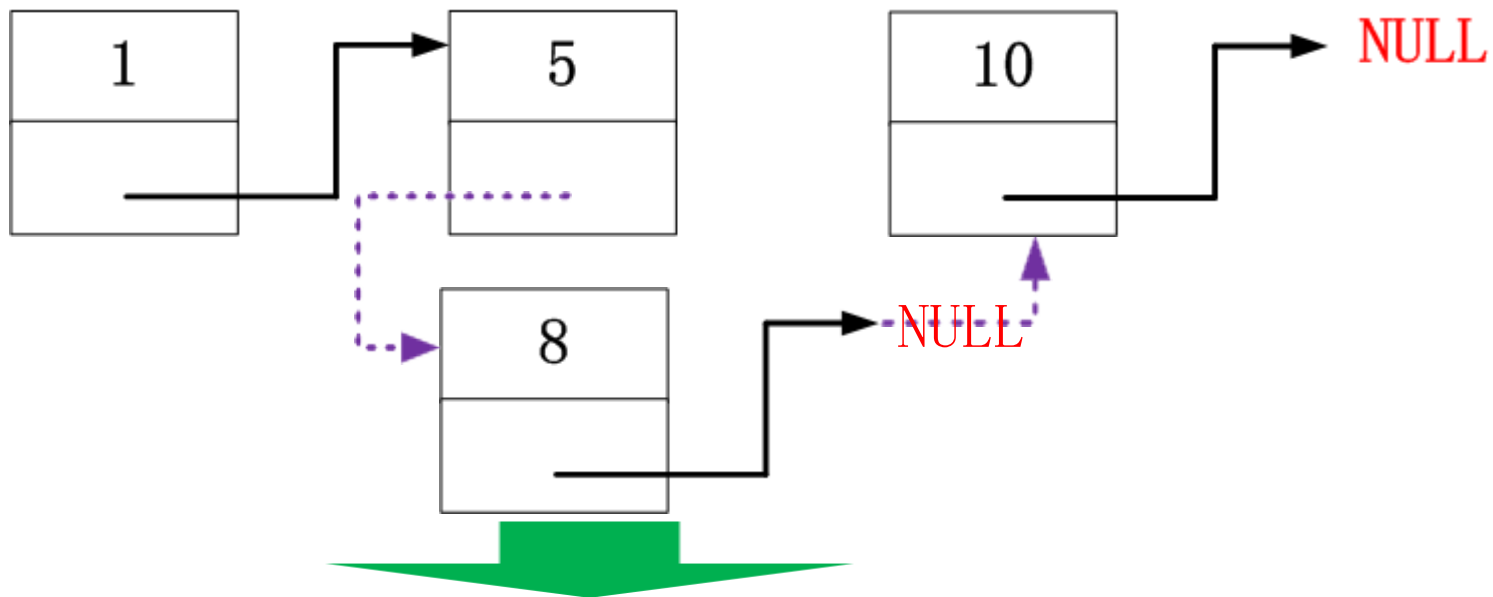
建立临时指针

```
Node *curr = head;
```

通过循环，由头指针所指地址，移动到尾指针所指地址，每移动到一个节点访问该节点保存的数据

```
while (curr != NULL) {  
    cout << curr->num; //访问数据  
    curr = curr->next; //移动到下一个  
}
```

插入链表节点



节点插入成员函数

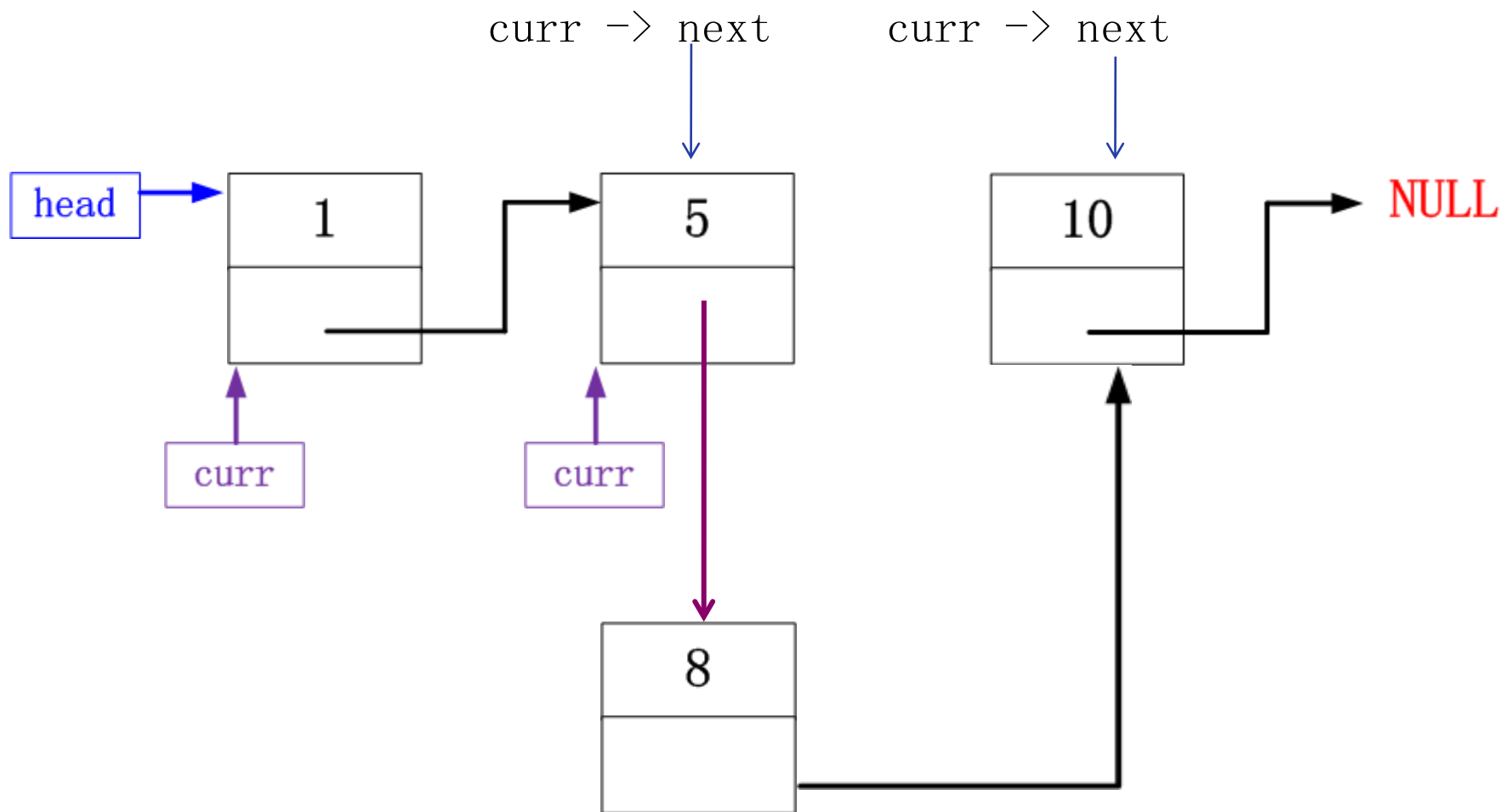
- 实现链表节点的有序插入（由小到大）

```
void List::Insert(int n)
{
    Node* tmp = new Node(n);
    //建立并初始化待插入节点
    if (head == NULL) {
        //链表头指针为空，即链表未建立
        head = tail = tmp;
        nodeCount++;
    }
```

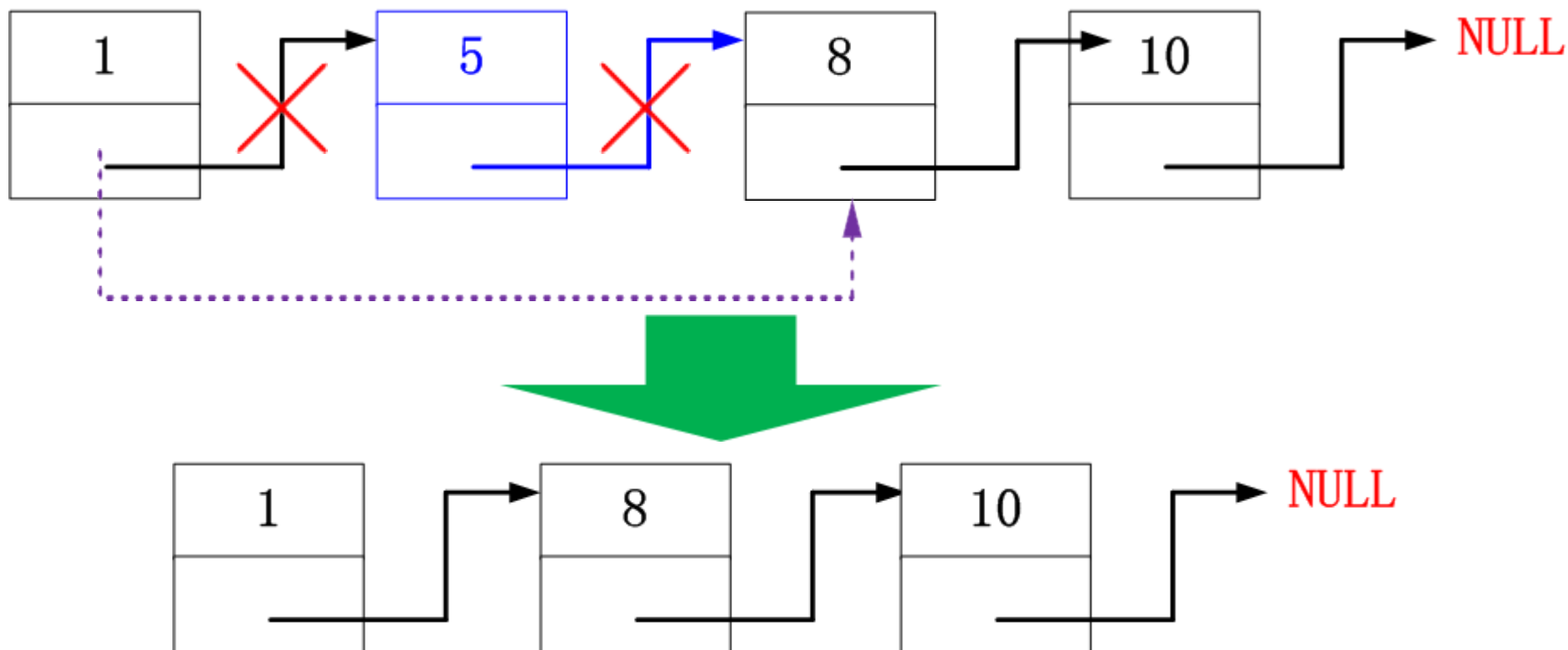
```
else
{
    if (n<head->num) //当前数据小于链表头结点保存的数据
    {
        tmp->next = head;
        head = tmp;
        nodeCount++;
        return; //为什么return?
    }
    if (n>tail->num) //当前数据大于链表尾结点保存的数据
    {
        tail->next = tmp;
        tail = tmp;
        nodeCount++;
        return;
    }
}
```

```
Node* curr = head;
while (curr->next != NULL) //寻找合适的插入位置
{
    if ((curr->num <= n) && (curr->next->num > n))
    {
        tmp->next = curr->next;
        curr->next = tmp;
        nodeCount++;
        return;
    }
    curr = curr->next;
} //end while
} //end else
} //end function
```

插入节点过程



删除链表节点



查找链表节点

在链表节点遍历过程中，判断当前节点的数据与待查找数据是否相同，相同则输出该节点位置

遍历到达尾指针时，链表仍然没有符合要求的数据，则输出“节点未找到”

主函数中进行链表的创建以及对链表进行各类操作

```
void main() {  
    List list;  
    int count;  
    cout << endl << "Please input the count of the  
node of the list: ";  
    cin >> count;  
    srand(time(0));  
    //生成随机数, 作为链表节点保存的数据  
    for (int i = 1; i <= count; i++) {  
        int tmp = rand() % 100;  
        list.Insert(tmp);  
    }  
    list.Print();  
}
```

```
list.Print();
int number;
cout << "Please input the number of the node you
want to insert: ";
cin >> number;
list.Insert(number);
list.Print();
cout << endl << "Please input the number of the
node you want to delete: ";
cin >> number;
list.Remove(number);
list.Print();
cout << endl << "Please input the number of the
node you want to search: ";
cin >> number;
list.Find(number);
list.Print();
}
```

练习7.3

随机生成n个整数，按照节点数据由小到大的顺序构造有序链表，完善节点类和链表类的如下成员函数：

- 成员函数：Remove(),链表节点删除
- 成员函数：Find(),链表节点查找
- 成员函数：Print(),按顺序输出链表全部节点的数据项
- 其它未定义成员函数，包括节点类的构造函数与析构函数、链表类的构造函数

栈 (Stack)

栈是程序设计过程中经常使用的一种**数据结构**，它对于数据的存放和操作有下面这样的特点：

- 它只有一个对数据进行存入和取出的端口；
- 后进者先出，即最后被存入的数据将首先被取出。其形式很像一种存储硬币的小容器，每次只可以从顶端压入一个硬币，而取出也只可从顶端进行，即后进先出。

这样的数据存储和管理形式在一些实际的程序设计中很有用

设计栈类

栈中需要保存一批同类型的数据，可用一个数组来实现。对数据的操作总从栈顶进行，需要设一个变量记录栈顶位置。从而抽象两个数据成员：

```
float data [maxsize];  
//data中存放栈的实际数据  
int top;    //top为栈顶位置
```

设计栈类

考虑对栈中数据要施加的操作，而抽象出其成员函数：

```
void push (float a);  
//将数据a “压入” 栈顶  
float pop (void);  
//将栈顶数据 “弹出” 并返回  
bool empty(void);  
//判断栈是否为空
```

设计栈类

为了显示对象的“诞生”与“死亡”信息, 还可对所设计的栈类加入构造与析构函数。

main函数中说明stack类型的对象s1、s2来表示两个具体的栈, 进而通过类成员函数对那两个具体的栈进行不同的操作。

```
//Stack.h
```

```
const int maxsize = 6;  
class Stack {  
    float data[maxsize];  
    int top;  
public:  
    Stack(void) ;  
    ~Stack(void) ;  
    bool empty(void) ;  
    void push(float a) ;  
    float pop(void) ;  
};
```

```
//Stack.cpp
#include "Stack.h"
#include <iostream>
using namespace std;

Stack::Stack(void) {
    top = 0;
    cout << "stack initialized." << endl;
}

Stack::~~Stack(void) {
    cout << "stack destroyed." << endl;
}

bool Stack::empty(void) {
    return top == 0 ? true : false;
}
```

```
void Stack::push(float a) {
    if (top == maxsize) {
        cout << "stack is full!" << endl;
        return;
    }
    data[top] = a;
    top++;
}

float Stack::pop(void) {
    if (top == 0) {
        cout << "Stack is underflow!" << endl;
        return 0;
    }
    top--;
    return data[top];
}
```

```
//Main.cpp
#include "Stack.h"
#include <iostream>
using namespace std;

void main() {
    Stack s1, s2;
    int i;
    for(i = 1; i <= maxsize; i++) s1.push(2 * i);
    for(i = 1; i <= maxsize; i++) cout << s1.pop() << " ";
    for(i = 1; i <= maxsize; i++) s1.push(2.5*i);
    for(i = 1; i <= maxsize; i++) s2.push(s1.pop());
    cout << endl;
    do
        cout << s2.pop() << " ";
    while (! (s2.empty()));
    cout << endl;
}
```

程序运行结果：

stack initialized.

stack initialized.

12 10 8 6 4 2

2.5 5 7.5 10 12.5 15

stack destroyed.

stack destroyed.

队列 (Queue)

队列 (Queue)是另一种**线性数据结构**，它只允许在表的一端插入元素，而在另一端删除元素，所以队列具有**先进先出**(Fist In Fist Out, 缩写为FIFO)的特性

- 在队列中，允许插入的一端叫做队尾(rear)，允许删除的一端则称为队头(front)。
- 假设队列为 $q=(a_1, a_2, \dots, a_n)$ ，那么 a_1 就是队头元素， a_n 则是队尾元素。队列中的元素是按照 $a_1, a_2, \dots, a_n$ 的顺序进入的，退出队列也必须按照同样的次序依次出队，也就是说，只有在 $a_1, a_2, \dots, a_{n-1}$ 都离开队列之后， a_n 才能退出队列。

队列的实现方式

顺序队列

- 基于数组结构，存储队列元素
 - 存储空间受限制
 - 删除操作不易实现

链式队列

- 基于链表结构，存储队列元素
 - 存储空间不受限制
 - 插入删除操作比较容易

节点类和队列类

```
class Qnode {  
public:  
    int data;  
    Qnode *next;  
};
```

```
class LQueue {  
public:  
    Qnode *front;  
    Qnode *rear;  
    void initQueue();  
    int empty();  
    void push(int);  
    void pop();  
    int getFront();  
    int getRear();  
    int getSize();  
    void printQueue();  
};
```

```
void LQueue::initQueue() {
    front = rear = new Qnode();
    front->next = rear;
    rear->next = nullptr;
}

int LQueue::empty() { // front = rear
    return front->next == rear;
}

void LQueue::push(int x) {
    Qnode *s = new Qnode();
    s->data = x;
    s->next = nullptr;
    rear->next = s;
    rear = s;
}
```

```
void LQueue::pop() {  
    if (empty()) {  
        cout << "LQueue is empty,can't pop.\n";  
        return;  
    }  
    front = front->next;  
}  
  
int LQueue::getFront() {  
    return front->next->data;  
}  
  
int LQueue::getRear() {  
    return rear->data;  
}
```

```
int LQueue::getSize() {
    Qnode *q = front->next;
    int k = 0;
    while (q!=nullptr) {
        k++;
        q = q->next;
    }
    return k;
}

void LQueue::printQueue() {
    Qnode *q = front->next;
    while (q) {
        cout << q->data << " ";
        q = q->next;
    }
    cout << "\n";
}
```

```
int main() {  
    LQueue L;  
    L.initQueue();  
    cout << "Push data to Queue...\n";  
    srand((unsigned)time(NULL));  
    for (int i = 0; i < 10; i++) {  
        L.push(rand() % 100);  
    }  
    cout << "Push finished.\n";  
    cout << "You have pushed such data:";  
    L.printQueue();  
    cout << "Pop data out of Queue...\n";  
    L.pop();  
    cout << "Pop finished.\n";  
    cout << "Now the Queue have such data:";  
    L.printQueue();  
    return 0;  
}
```

程序运行结果

Push data to Queue...

Push finished.

You have pushed such data:88 66 97 55 36 0 16 84 17 0

Pop data out of Queue...

Pop finished.

Now the Queue have such data:66 97 55 36 0 16 84 17 0

练习7.4

- (1) 编程实现链式栈和链式队列
- (2) 使用链表实现一元多项式的加法
- (3) 使用栈实现十进制数转换为二进制数
- (4) 自学标准模板库的list和stack类模板，编程实现 (2) 和 (3)

第七章 结束

